

Respuesta a la pregunta 5

Arquitectura de software

El firmware del sistema estará organizado mediante una arquitectura de Polling más interrupciones. Las interrupciones se utilizan en puntos críticos donde se requiere minimizar el error. Dado que el robot debe navegar por sí mismo, saber dónde está y corregir su movimiento, evitar que el error se acumule es crucial. El software está conformado por las siguientes capas:

- Drivers
- Estimación
- Navegación y control
- Aplicación

Drivers

Esta capa se encarga de configurar e inicializar los diferentes sensores y sistemas del vehículo y ofrecer una integración transparente entre el hardware y el resto del software. Aquí se incluyen los drivers de ambos motores, el driver de la unidad de medición inercial, el driver del sensor ultrasónico, driver del encoder de los motores y el driver de los sensores infrarrojos detectores de línea. La capa Drivers está conformada por los siguientes módulos:

- **Driver_TB6612FNG:** Conformado por los archivos **Driver_TB6612FNG.c/Driver_TB6612FNG.h**, implementa las funciones que permiten realizar las siguientes tareas:
 - Implementar las funciones de control de velocidad.
 - Dirección del giro del motor.
 - Generación del PWM necesario para controlar la velocidad.
 - Frenar el motor.
- **IMU:** Conformado por los archivos **imu.c / imu.h**, implementa las funciones que permiten configurar, inicializar y utilizar la unidad de medición inercial (IMU). Las tareas de este módulo son:
 - Inicializar la comunicación I2C
 - Configurar los registros internos del MPU-6050.
 - Realizar calibración.
 - Leer los datos crudos medidos por el acelerómetro y giroscopio.
 - Establecer los rangos de las mediciones.
 - Procesar las mediciones y transformarlas en unidades físicas utilizables por el sistema.
- **Ultrasonic:** Conformado por los archivos **ultrasonic.c / ultrasonic.h**, implementa las funciones que permiten utilizar el sensor ultrasónico HC-SR04. Sus tareas son:
 - Gestionar la temporización del sensor ultrasónico.
 - Generar los pulsos de disparo del sensor.
 - Medir la duración del eco.
 - Estimar de la distancia.

Este módulo emplea interrupciones de temporizador (TIMER_IRQ) para medir con precisión la duración de los pulsos y estimar correctamente la distancia.

- **Encoder:** Conformado por los archivos `encoder.c` / `encoder.h`, se encarga de determinar las revoluciones y el recorrido de cada rueda. Sus tareas son:
 - Contar los pulsos por medio de las interrupciones GPIO
 - Implementar una temporización periódica para el cálculo de velocidad angular.
 - Estimar las RPM del motor.
 - Estimar el desplazamiento del robot.
 - Filtrar las lecturas de ser necesario.
- **Infrarrojo:** Conformada por los archivos `infrarrojo.c` / `infrarrojo.h`, permite ejecutar las tareas relacionadas con la lectura de los cuadrantes del área de trabajo y el límite de esta además de identificar el color de la caja. Dado que el robot posee dos sensores infrarrojos con tareas distintas, se aclara que cada sensor tiene sus funcionalidades específicas:
 - Sensor inferior: Detectar los límites del área de trabajo.
 - Sensor frontal: Determinar el color de la caja cuando el robot se acerque a ella.

La información de estos sensores les indica a las capas superiores información relevante para la corrección del mapa del entorno.

Estimación

Esta capa se encarga de realizar la odometría y la fusión sensorial. Se comunica con la capa de drivers, de donde obtiene los datos necesarios para saber en donde se encuentra el robot. Esta capa le da la capacidad al robot de responder a las preguntas ¿Dónde estoy? ¿Qué hay en el entorno? ¿Cómo me estoy moviendo? Por medio de la integración de las distintas métricas. Esta capa es un único módulo formado por los archivos `estimacion.c` / `estimacion.h`, Las funciones del módulo son las siguientes:

- Realizar la odometría del robot
- Obtener los datos de la unidad de medición inercial.
- Fusionar los datos de la IMU con la odometría reduciendo el efecto de deriva.
- Detectar las cajas por medio de la lectura del sensor ultrasónico.
- Determinar el color de la caja.
- Estimar la posición del robot en el área de trabajo tanto en (x, y) como angularmente.
- Estimar la velocidad del robot.
- Determinar la distancia a la caja.
- Generar el mapa virtual del entorno.

Navegación y control

Esta capa se encarga de controlar el movimiento del robot y planificar su trayectoria por medio de distintas métricas proporcionadas por la capa de Estimación. Esta capa la conforman dos módulos `control.c` / `.h` y `navegacion.c` / `.h`

- **Control:** Sus funciones son:
 - Realizar el control de bajo nivel de la velocidad y dirección del motor.
 - Implementar el control PID de los motores.

- Corregir las desviaciones en la trayectoria.
- **Navegación:** Sus funciones son:
 - Calcular la ruta hacia las cajas detectadas.
 - Ignorar las cajas que ya han sido identificadas por su color.
 - Coordinar las maniobras de aproximación y empuje.
 - Detectar la salida del área de trabajo y corregir la ruta para reingresar a él.
 - Generar referencias de orientación y velocidad para el robot.

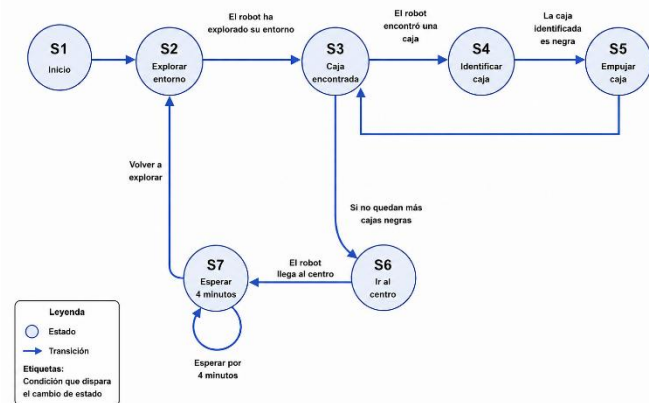
Aplicación

Esta capa integra la aplicación del robot. Define el comportamiento específico a nivel de usuario, dictando que hace y que no hace el robot. El robot busca cajas dentro de su entorno de trabajo y extrae de él las cajas de color negro e ignora las cajas de otro color. Cuando detecta a distancia una caja, si no la ha identificado antes, se acerca a ella y la identifica. Si la caja identificada es negra, la expulsa fuera del área de trabajo. Una vez no hallan más cajas negras, el robot se situará en el centro del área de trabajo y volverá a leer el entorno pasados 4 minutos. Está conformada por el módulo **aplicación.c/.h**, cuyas funciones son:

- Tomar las decisiones de alto nivel por medio de la máquina de estados general del robot.
- Coordinar las capas inferiores.
- Implementar el comportamiento general del robot cumpliendo así con los requisitos funcionales propuestos.

La Máquina de estados tiene por estados:

- **S1 = APP_INIT:** El robot inicializa.
- **S2 = APP_EXPLORE:** El robot explora su entorno y crea un mapa virtual de él.
- **S3 = APP_APPROACH_BOX:** El robot busca cajas en la distancia y las detecta.
- **S4 = APP_IDENTIFY_BOX:** El robot identifica la caja que tiene en frente.
- **S5 = APP_PUSH_BOX:** Expulsa la caja identificada si es de color negro.
- **S6 = APP_CENTER:** El robot regresa al centro del área de trabajo cuando ya no queden cajas negras.
- **S7 = APP_WAIT:** El robot espera durante un tiempo definido para volver a explorar.



Esquema de la máquina de estados finitos de la aplicación.

Dado que el sistema posee una arquitectura basada en Polling + Interrupciones, las interrupciones son empleadas en aquellos puntos críticos donde se requiere reacción rápida por parte del sistema. Las interrupciones más importantes utilizadas son:

Interrupciones GPIO

Empleadas por los encoders de los motores, se utilizan para detectar los flancos de pulso. Cada interrupción incrementa o decrementa el conteo de los pulsos necesarios para la odometría. También es utilizado por el driver del sensor infrarrojo.

Interrupciones de TIMING

Empleadas principalmente por el sensor ultrasónico para medir correctamente la distancia y por el módulo Driver_TB6612FNG para realizar el envío periódico de la velocidad y dirección. También se utilizarán interrupciones de timing e interrupciones periódicas en secciones de la lógica que requieran manejo del tiempo.